NEBIUS X NVIDIA GLOBAL AI HACKATHON · CODING AND AGENTIC ENGINEERING TRACK

ShipCheck

Product Requirements Document

Adversarial verification gate for AI-generated code — attacks code in Token Factory Sandboxes, returns a verdict with reproducible exploit demos, plus a rectification CD loop: find, fix, re-verify, PR for a human. Never auto-merge.

V1.0 LOCKED FOR BUILD TEAM OF 3

Field	Detail
Version	1.0 · October 4, 2026 · Status: Locked for build
Hackathon	Nebius x NVIDIA Global AI Hackathon — Coding and Agentic Engineering track
Team	3 builders
Submission deadline	**October 30, 2026, 10:00 AM PT**
Judging	December 1–15, 2026 (demo must stay live through judging)
One-line pitch	“Don’t trust AI-generated code — attack it. ShipCheck proves vulnerabilities with working exploits, fixes them, proves the fix, and hands you the PR.”

> **How to use this PRD.** This is the build reference. Sections 5 and 6 are the contract: FR-1 through FR-8, the component diagram, exact model IDs, and the four critical integration gotchas. If a choice is not in here, check §13 Open Questions before inventing one.

CONTENTS

Table of contents

§	Section	Page
1	Overview — problem and product	3
2	Goals and Non-Goals	4
3	Users and Personas	4
4	Product Scope — MVP	5
5	Functional Requirements FR-1 through FR-8	6–8
6	Technical Architecture — diagram, model tiers, gotchas	9–10
7	Non-Functional Requirements	10
8	Milestones — 4-Week Build Plan (3 people)	11
9	Risks and Mitigations	12
10	Demo Script (3 minutes)	13
11	Submission Checklist	13
12	Judging-Criteria Mapping	14
13	Open Questions / Verify on Day 1	14
A	Appendix A — Key Research Sources	15

Document control

This PRD is faithful to the locked ShipCheck scope: adversarial verification gate for AI-generated code in Token Factory Sandboxes, verdict with reproducible exploit demos, and the rectification CD loop (find → fix → re-verify → PR for a human, never auto-merge). Model IDs, API patterns, sandbox commands, and marker rules are reproduced exactly as verified in the draft — copy them case-sensitively; a typo is a 404.

> **Deadline drives scope.** With submissions closing **October 30, 2026 at 10:00 AM PT**, the MVP is deliberately narrow: PR-diff scope only, four vulnerability classes, Python and JavaScript/TypeScript. Everything else is an explicit non-goal (§2) or a stretch.

Conventions

- **Must / never** language is normative. “Never auto-merge” and “never call `PUT /pulls/{n}/merge`” are hard constraints, not preferences.
- Code blocks and inline commands are copy-paste contracts. Do not paraphrase flags or model IDs.
- “Confirmed” means a sandbox PoC marker fired (§5 FR-4). LLM judgment alone never confirms a finding.

SECTION 1

1. Overview

Problem

AI coding assistants now generate a large share of production code, but nobody independently verifies it. Published studies put vulnerability rates in AI-generated code at 25–45%; AI-assisted repos carry ~4.4x more vulnerabilities than human-only ones; and a passing test suite generated by the same agent that wrote the code provides no independent assurance — agents game green CI. The industry's own answer (Google CodeMender, GA October 2026) validates the loop ShipCheck implements: find → prove exploitability with a real PoC in an isolated sandbox → fix → human approves, never auto-commit.

Product

ShipCheck is an **adversarial verification gate** for AI-generated code. Given a repository or PR diff, it:

1. Plans attacks with a reasoning model (Nemotron 3 Ultra/Super),
2. Executes them as parallel probes inside disposable Nebius Token Factory Sandboxes — property-based tests, mutation probes, dependency/secret scans, and **generated proof-of-concept exploits** (SQLi, command injection, path traversal, reflected XSS),
3. Returns a **verdict report** with a risk score and reproducible exploit demos,
4. **Rectifies**: generates a fix per confirmed finding, re-runs the attack to prove the exploit is closed, runs the repo's own tests, and opens a **draft GitHub PR for a human reviewer — never auto-merge**.

> **One-line pitch**
>
> "Don't trust AI-generated code — attack it. ShipCheck proves vulnerabilities with working exploits, fixes them, proves the fix, and hands you the PR."

Why this wins the track

The track rewards agents that write, run, and test code inside Token Factory Sandboxes. ShipCheck uses the sandbox for its most defensible purpose — disposable, parallel, localhost-only exploit execution with checkpoint branching for fix verification — and pairs it with tiered Nemotron models where each tier has a distinct job (§6). The demo is falsifiable: the exploit lands, then it does not.

2. Goals and Non-Goals

Goals (must achieve by Oct 30)

- **G1.** End-to-end pipeline on fixtures: ingest → attack plan → PoC probes → verdict → fix → re-verify → draft PR.
- **G2.** Deterministic, watchable 3-minute demo: exploit lands → fix generated → attack re-run fails → PR opened.
- **G3.** Public demo URL (Nebius Serverless Endpoint), public repo with OSS license, ≤3-min YouTube video with audio covering Token Factory + Nemotron usage.
- **G4.** Stackable $3,000 “Best Use of Tavily” prize: Tavily grounds every confirmed finding (CVE/advisory IDs, affected versions, canonical fix) in the verdict report and PR body.

Non-goals (explicitly out of scope for the hackathon build)

- Full-repo scans (PR-diff scope only; async full scans are stretch).
- Vulnerability classes beyond the MVP four: SSRF, crypto flaws, auth logic, race conditions, deserialization beyond pickle, C/memory-corruption.
- Mutation testing as a quality gate (stretch probe only).
- MCP-server orchestration (use `contree-sdk` directly).
- Auto-merge under any circumstances.

3. Users and Personas

Persona	Need
P1 — The skeptical tech lead	“An agent wrote 2,000 lines this sprint. Prove to me it’s safe before I merge.” Wants the verdict report + risk score.
P2 — The PR reviewer	Wants the draft PR body to contain everything needed to approve: PoC evidence, root cause, fix diff, re-run result, reviewer checklist.
P3 — The hackathon judge	Watches a 3-minute video. Needs a visceral, falsifiable demo and a clear story of why Token Factory Sandboxes + Nemotron were load-bearing.

Design implication: the verdict report serves P1, the PR body serves P2, and the live probe view serves P3. All three are rendered from the same verdict JSON (§5 FR-5, FR-8).

SECTION 4

4. Product Scope — MVP

Vulnerability classes (demo-reliability order)

#	Class	Why it is in the MVP and how the fix reads
1	**SQL injection**	LLMs flag it reliably; fix is stereotyped (parameterized queries); PoC deterministic (extra rows returned).
2	**Command injection**	Same profile; fix is `subprocess.run([...], shell=False)` + input allowlist; `uid=` marker unambiguous.
3	**Path traversal**	LLMs miss it when code "looks clean" (`os.path.join(BASE_DIR, filename)` idiom); the PoC (actual file read) is the detection backstop — this is the ShipCheck thesis in one demo beat. Fix: resolve + prefix check.
4	**Reflected XSS**	Reflection proof needs no browser (raw HTML contains unescaped payload); fix is `html.escape` / template autoescape.

Languages

Python and JavaScript/TypeScript. (Java via PIT is a stretch; C is out.)

Input scope

A GitHub repo URL or PR diff. Probes run against the PR diff / changed files vs `main` — never the whole tree.

> **Scope is the safety mechanism.** Four stereotyped classes with objective markers are what make the rectification loop credible in four weeks. The research base rate is sobering — only 26% of LLM security patches fix the vuln without changing behavior (§9) — so ShipCheck answers with re-verification and a human gate rather than broader coverage.

What "confirmed" means

A finding is confirmed only when its PoC marker fires in a sandbox (§5 FR-4). Static triage and LLM planning prioritize and explain; they do not confirm. Unconfirmed findings still appear in the report with exploitability ~0 until a marker proves otherwise.

5. Functional Requirements

FR-1 · Ingest

- Accept a repo URL (clone) or PR diff; materialize code into the pre-baked `shipcheck-probe` sandbox image (Python/Node, Semgrep, gitleaks, pip-audit, osv-scanner preinstalled; checkpointed once, reused).
- Output: immutable base image UUID for the scan.

FR-2 · Static triage (cheap → expensive)

Run in order; SAST findings feed the attack planner:

1. `gitleaks detect --redact --report-format sarif` (tree + history).
2. `pip-audit --format json` / `npm audit --omit=dev --audit-level=high --json` fast pass; `osv-scanner scan source --format sarif` for transitive CVEs; OSV.dev API for per-package CVE details.
3. `semgrep scan --config p/owasp-top-ten --sarif` (+ framework packs).
4. Each probe emits `{probe, target, verdict, evidence}` JSON.

FR-3 · Attack planning (Nemotron 3 Ultra)

- Input: diff + triage findings (full context; Ultra's 1M context).
- Output: attack plan JSON — list of `{vuln_class, target_file, sink, payload_strategy, expected_marker}`.
- Tavily powers the planner with current exploit writeups for the detected stack (`TavilyClient.search`, `include_answer=True`).

Planning is the one place Ultra's 1M context is load-bearing: the whole diff plus triage can be reasoned over at once. Everything downstream is mechanical execution against the plan.

5. Functional Requirements (continued)

FR-4 · PoC exploit probes (parallel sandboxes)

- One sandbox branch per finding: fork from base image, app + attacker in the **same** microVM run chain (localhost-only — no egress dependency).
- Fan-out via `asyncio.gather` behind an `asyncio.Semaphore(40)` (beta cap is 50 concurrent ops).
- Each probe: start target → send crafted payloads → capture single JSON evidence blob `{payload, request, response_status, response_excerpt, verdict, marker_found}` + raw logs as downloadable artifacts.
- **Verdict rule is boolean on an objective marker**, never on LLM judgment:

Class	Payload → marker that confirms
SQLi	Payload `' OR 1=1--` → response returns >1 user / admin email present.
Command injection	`8.8.8.8; id` → stdout contains `uid=`.
Path traversal	`../../etc/passwd` → response contains `root:`.
Reflected XSS	`?q=<script>alert(1)</script>` → raw HTML contains the unescaped payload.

- Client-side timeouts on every probe; fixed port allocation per probe; deterministic fixtures as the demo path.

FR-5 · Verdict report + risk score

- Per-finding card: severity band, exploitability (sandbox-PoC-confirmed = hard escalator ×1.5–2.0; unexploitable → ~0), reachability (sink reachable from an entrypoint?), exposure (network service vs library). **One sentence per factor** explaining "why is this #1".
- Risk score 0–100, capped-points model. Confirmed false positive → 0.
- Report rendered as JSON → Markdown → dashboard; every confirmed finding cites Tavily-grounded CVE/advisory IDs, affected versions, canonical fix.

> **Deterministic closure.** The verdict is a fact (marker present or absent), not an interpretation. That is what makes the 3-minute demo watchable and the rectification loop (§5 FR-6) gradeable.

5. Functional Requirements (continued)

FR-6 · Rectification (fix → re-verify)

- Fix generator (Nemotron Super/Ultra) receives the **exploit evidence, not just the code** (LLMs miss clean-looking traversal code).
- Model outputs **diff-hunk JSON**, applied programmatically via search/replace (Nemotron Super's structured patch outputs are reliable enough for mechanical application).
- Re-run the PoC in a **child branch**: marker must be absent (green) **and** the repo's own test suite must pass (CodeMender pattern).
- If the fix fails re-verification or breaks tests: report "fix failed verification" — never silently downgrade.

FR-7 · GitHub PR (never auto-merge)

- Bot flow: apply patch in sandbox → `git push origin shipcheck/fix-<class>-<slug>` → `POST /repos/{owner}/{repo}/pulls` with `{title, head, base, body, draft: true}` → 201.
- Dedupe: `GET /pulls?state=open`, match `head.ref` before creating.
- Auth: fine-grained PAT with **Pull Requests: write only** (least privilege is a judging talking point). **Never call `PUT /pulls/{n}/merge`.**
- PR body template (the demo artifact): severity/risk score → PoC evidence (payload + observed marker + sandbox run/artifact link) → root cause → fix diff → re-run result (exploit blocked, tests green) → reviewer checklist.

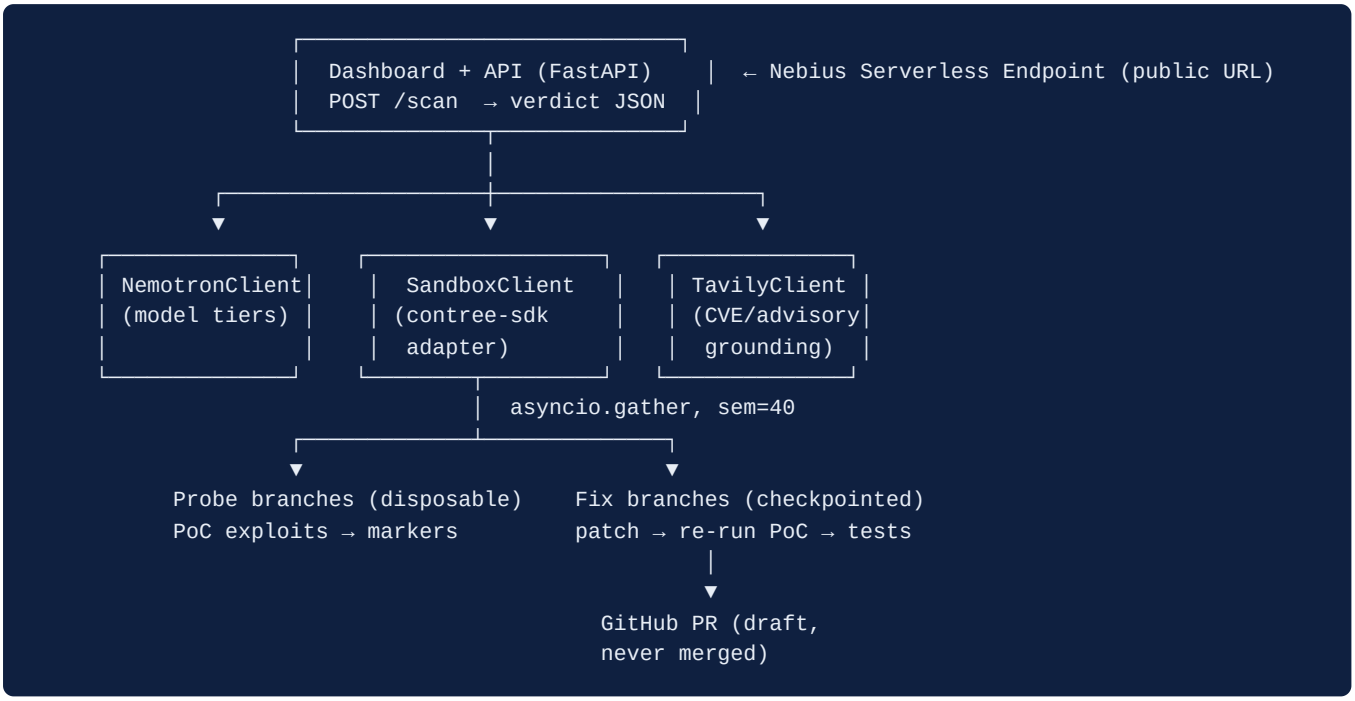
FR-8 · Dashboard / API (public demo URL)

- Single-page UI served by a FastAPI app on a Nebius Serverless Endpoint (CPU preset is fine; inference stays on Token Factory).
- Views: scan input → live probe status → verdict report with risk-ranked findings → per-finding evidence (payload, marker, logs) → fix diff + re-verification result → PR link.
- Keep the app stateless (replicas are ephemeral); warm the URL before recording (cold starts).

SECTION 6

6. Technical Architecture

Component map



Model tier mapping

Tier	Model ID	Role
Ultra	`nvidia/Nemotron-3-Ultra-550b-a55b` (1M ctx)	Attack planning over full context; hard-vuln patch synthesis. Budget per scan.
Super	`nvidia/nemotron-3-super-120b-a12b` (256K)	Orchestrator/supervisor; PoC exploit script generation; patch review.
Nano	`nvidia/NVIDIA-Nemotron-3-Nano-30B-A3B` (262K)	Log compaction, pass/fail parsing, evidence summarization, Tavily query synthesis.
Lightning	`nvidia/Nemotron-3_5-Lightning` (1M, fast)	High-throughput JSON extraction loops.

Base URL: `https://api.tokenfactory.nebius.com/v1/` (OpenAI-compatible; regional fallback `https://api.tokenfactory.us-central1.nebius.com/v1/`). Copy model IDs case-sensitively — a typo is a 404.

Sandbox base URL

`https://api.tokenfactory.nebius.com/sandboxes` · `pip install contree-sdk` (`Contree` / `ContreeSync` clients; `images[0].run(shell=...)` with `.wait()` / `await`; `disposable=false` → immutable checkpoint images; N runs from one image = branches).

6. Technical Architecture (continued)

Critical integration gotchas (load-bearing)

1. **Do not architect around OpenAI function-calling.** On Nebius, Nemotron reasoning models may return empty `message.content` with the answer in `reasoning_content`, and function-calling through the wrapper is unconfirmed. Reliable pattern: JSON-in-prompt + last-JSON-block extraction + `extra_body={"chat_template_kwargs": {"enable_thinking": False, "force_nonempty_content": True}}` + generous `max_tokens`. Read `content or reasoning_content`. Wrap in one `NemotronClient` with retries.
2. **Pin `contree-sdk`** and isolate all SDK calls in a `SandboxClient` adapter (documented 0.3.6 constructor drift vs docs; IAM auth needs `NEBIUS_PROJECT_ID`).
3. **Localhost-only PoCs.** Sandbox outbound network is unconfirmed — app + attacker live in the same microVM run chain; pre-bake all dependencies into the checkpoint image; never `pip install` at probe time. (Note: process memory isn't preserved across runs, so start-server → attack → capture must be one `run(shell=...)` chain.)
4. **Beta cap:** 50 concurrent ops → `asyncio.Semaphore(40)`; per-run resource limits unpublished → short, bounded probes with client-side timeouts.

7. Non-Functional Requirements

- **NFR-1 · Scan latency:** PR-diff scan completes in minutes, not tens of minutes (probe parallelism; diff-scoping).
- **NFR-2 · Determinism:** the demo path (seeded fixtures) must be green/red on rails — no flaky ports, no timing-dependent markers.
- **NFR-3 · Cost:** verify Nemotron per-token pricing on the Token Factory console before budgeting; keep Ultra calls bounded (planning + hard fixes only).
- **NFR-4 · Safety of the tool itself:** tokens via env vars only; `--redact` on all secret scans; never log credentials; fine-grained GitHub PAT (PRs: write, no merge scope); PoCs run only in isolated sandboxes against test fixtures.
- **NFR-5 · Availability:** demo URL live and warm through judging (Dec 1–15, 2026).

SECTION 8

8. Milestones — 4-Week Build Plan (3 people)

Week	P1 — Sandboxes & probes	P2 — Models & loop	P3 — Surface & ship
W1 · Plumbing	`SandboxClient` (spawn/run/poll/artifacts, semaphore, timeouts) + pre-baked probe image	`NemotronClient` (JSON-extraction fallback, role presets) + attack-planner prompt	Verdict report renderer (JSON → Markdown → minimal dashboard)
Milestone: One hardcoded SQLi fixture → probe → verdict JSON end-to-end			
W2 · The loop	4 PoC exploit probes (SQLi, CMDi, traversal, XSS) with marker verdicts	Fix generator (diff-hunk JSON) + re-run-verifies-closed + regression-test gate	GitHub integration: push branch, evidence-rich PR body, draft PRs
Milestone: Exploit lands → fix → re-run fails → PR opened, on fixtures			
W3 · Score, ground, deploy	Risk-score engine + Tavily CVE grounding in reports/PRs	Deploy dashboard + API to Nebius Serverless Endpoint (public demo URL)	Demo repo curation + 3-min video script/rehearsal; README (model IDs, where TF/Nemotron/Tavily accelerated the work)
W4 · Harden & submit	All: real-repo shakedown, flakiness fixes, judging-criteria pass (Implementation / Design / Impact / Idea), public repo + OSS license + YouTube ≤3 min with audio covering Token Factory & Nemotron + written feedback on Nebius/NVIDIA tooling. Buffer for sandbox/IAM surprises.		

Role split rationale: P1 owns the flakiest surface (beta sandbox SDK) behind one adapter; P2 owns the model contract (one `NemotronClient`, tier presets, prompts); P3 owns everything a judge touches (dashboard, PR body, video). No cross-blocking after W1 contracts (verdict JSON, attack-plan JSON) are frozen.

SECTION 9

9. Risks and Mitigations

Risk	Mitigation
Nemotron function-calling absent / empty `content`	JSON-in-prompt + `reasoning_content` fallback + `force_nonempty_content`; single `NemotronClient` wrapper
`contree-sdk` API drift	Pin version; `SandboxClient` adapter isolating all SDK calls
No sandbox outbound network	Localhost-only PoCs; pre-baked checkpoint image; no installs at probe time
50 concurrent ops cap	Semaphore(40); short bounded probes; client-side timeouts
PoC flakiness (ports, timing)	Fixed ports, readiness polling, deterministic markers, fixtures as demo path
Fix breaks functionality	Re-run repo's own tests in the fixed branch before PR; draft PRs; never merge
Secret handling	Env-only tokens; `--redact`; fine-grained PAT (PRs: write only)
Only 26% of LLM patches are clean	Scope to the 4 stereotyped classes; re-verification loop + human gate are the product's answer

> **The 26% stat is the pitch, not the objection.** Off-by-1 Labs found only 26% of LLM security patches fix the vuln without changing behavior, and 10.3% pass all functional tests yet remain exploitable. ShipCheck exists because AI patches are unreliable; its re-verification loop and human gate are the answer to the exact study a skeptic would cite.

10. Demo Script (3 minutes)

Time	Beat
0:00–0:30	**The problem.** “AI wrote this login handler. Tests pass. CI is green.” Show the code.
0:30–1:15	**The attack.** ShipCheck plans and fires PoC probes in sandboxes. SQLi payload `' OR 1=1--` returns the admin row — marker on screen, verdict: CONFIRMED EXPLOITABLE, risk score.
1:15–2:00	**The fix.** Patch generated from the exploit evidence; applied in a child branch; **same attack re-run → marker absent**; repo tests green.
2:00–2:45	**The handoff.** Draft PR opens with the evidence body: PoC, root cause, diff, re-run result, reviewer checklist. “ShipCheck never merges — the human does.”
2:45–3:00	**The stack.** “Built on Nebius Token Factory Sandboxes, NVIDIA Nemotron 3 (Ultra plans, Super executes, Nano compacts), Tavily CVE grounding.”

11. Submission Checklist

- ☐ Public repo (GitHub) with OSS license (Apache-2.0 / MIT / MPL-2.0) at top level
- ☐ README: setup instructions, how to run, which NVIDIA models used, where Token Factory accelerated the work, where Tavily was used
- ☐ Working demo URL live (warm it before judging)
- ☐ Public YouTube ≤3 min, audio covering Token Factory + Nemotron usage
- ☐ Written feedback on Nebius Token Factory / AI Cloud / NVIDIA tooling
- ☐ Deadline: **Oct 30, 2026, 10:00 AM PT**

12. Judging-Criteria Mapping

Criterion	How ShipCheck scores
Technological Implementation	Disposable-sandbox attack runs with real telemetry; tiered Nemotron roles (Ultra/Super/Nano/Lightning); the sandbox API used in its most defensible way
Design	Complete gate: ingest → attack → verdict → fix → re-verify → PR; coherent dashboard; evidence-first PR body as the product surface
Potential Impact	25–45% vuln rates in AI-generated code; 10k+ new AI-code findings/month; every team shipping AI-written code is the audience
Quality of the Idea	Offense-as-a-service: nobody demos the attack landing *then closing*; sandbox-confirmed exploitability replaces guesswork

Differentiator vs Google CodeMender: CodeMender validates the loop shape (scan → sandbox PoC → fix → human approval) — ShipCheck is the **open-infrastructure, attack-first** version for teams outside GCP/Gemini enterprise contracts.

13. Open Questions / Verify on Day 1

1. Nemotron per-token pricing on Token Factory (verify on console before budgeting).
2. Function-calling on Nebius: test `tools=` on your account; default to the JSON-in-prompt pattern regardless.
3. Sandbox outbound network: test `pip install` / `git clone` from inside once; design assumes none.
4. Pin `contree-sdk` version after testing; confirm `Contree(config, *, base_url, token)` signature.
5. Confirm 50-concurrent-op behavior under load; tune the semaphore.

APPENDIX

A. Key Research Sources

- Nebius Token Factory Sandboxes docs (overview, SDK, SWE-agents pages) — verified live
- `nebius/contree-sdk`, `nebius/contree-mcp` (GitHub)
- NVIDIA token-factory-cookbook Nemotron model cards; model IDs verified on a real account Oct 2026
- Google Cloud blog: CodeMender (scan → sandbox PoC → fix → human approval)
- GitHub Copilot Autofix efficacy reports; Snyk on the human-gate norm
- 1Password Off-by-1 Labs “F.L.A.W.E.D.” (patch-quality study, Aug 2026)
- Sonatype reproduce → fix → re-verify → PR-for-human (Vista Equity hackathons)
- tavily-python (PyPI); OSV.dev API; Semgrep, gitleaks, osv-scanner, StrykerJS, Hypothesis docs

> *Unverified at PRD time: Nemotron per-token pricing; function-calling on Nebius (conflicting reports); sandbox per-run resource limits and egress; exact MCP tool count. Each is listed in §13 with a verification action.*

Build contracts to freeze in Week 1

Contract	Shape
Triage probe output	`{probe, target, verdict, evidence}` JSON (§5 FR-2)
Attack plan	List of `{vuln_class, target_file, sink, payload_strategy, expected_marker}` (§5 FR-3)
PoC evidence blob	`{payload, request, response_status, response_excerpt, verdict, marker_found}` + raw logs (§5 FR-4)
Fix output	Diff-hunk JSON applied via search/replace (§5 FR-6)
Verdict report	JSON → Markdown → dashboard; per-finding card + 0–100 risk score (§5 FR-5)

End of PRD v1.0. Locked for build — October 4, 2026. Submission deadline October 30, 2026, 10:00 AM PT. Judging December 1–15, 2026.